

SPI Master-Slave Communication (STM32 to Arduino)

Overview

This project establishes a high-speed, full-duplex communication link using a custom Serial Peripheral Interface (SPI) driver. It acts as the final stage of the system, transmitting the formatted environmental data from the STM32 (Master) directly to an Arduino (Slave).

Objectives

- * Develop a bare-metal SPI driver supporting configurable CPOL, CPHA, and prescalers.
- * Manage the Shift Register to properly queue 8-bit Data Frame Formats (DFF).
- * Implement Software Slave Management (SSM) for manual Chip Select (NSS) control.
- * Configure the Arduino internal registers to act as an SPI Slave using interrupts.

Components Used

- * STM32 Nucleo Board (STM32F401RE / F446RE)
- * Arduino Uno (Configured as Slave)
- * Jumper Wires (SCK, MISO, MOSI, NSS, Common Ground)

Working Principle

SPI uses a dedicated clock line generated by the Master to synchronously shift data in and out. The driver configures PA5, PA6, and PA7 to connect to the internal SPI hardware. Before transmission, a standard GPIO pin (PA4) is pulled LOW to alert the Arduino. The STM32 then writes to the `SPI\_DR` register, which automatically pushes bits out on MOSI while the Arduino shift register catches them.

System Architecture

- * **STM32 (Master):** PA5 (SCK), PA6 (MISO), PA7 (MOSI) configured as AF5. PA4 configured as standard Push-Pull output for NSS.
- * **Transmission Protocol:** Software toggles NSS LOW, transmits a string byte-by-byte (including a `\0` terminator), then toggles NSS HIGH.
- * **Arduino (Slave):** MISO set to OUTPUT, `SPCR` register configured for Slave mode and SPI Interrupts.

Implementation Flow

1. Initialize SPI1 on the STM32 with ``SPI_CPOL_LOW``, ``SPI_CPHA_LOW`` (Mode 0), and a slow prescaler (``DIV128``).
2. Pull PA4 (NSS) LOW to select the Arduino.
3. Call ``SPI_Transmit_Buffer()`` to send the formatted temperature string.
4. Wait for the ``BSY`` (Busy) flag to clear, then pull PA4 HIGH.
5. On the Arduino, the ``SPI_STC_vect`` interrupt fires for every byte, assembling the string until the null terminator is hit, triggering a Serial print.

Key Observations

- * **Overrun Errors (OVR):** If the STM32 sends data faster than the Arduino can process the interrupts, data gets corrupted. Using a high clock prescaler (``DIV128``) was crucial for stability.
- * **Common Ground:** Without a shared ground wire between the STM32 and Arduino, the high-speed clock signals float, resulting in garbage data.

Results

A highly reliable, synchronous data bridge allowing the STM32 to offload the environmental data payload to a secondary Arduino microcontroller seamlessly.

Key Learnings

- * Understanding the physical mechanics of synchronous Shift Registers.
- * Hiding the complexity of ``TXE``, ``RXNE``, and ``BSY`` flag polling inside a unified Transmission API.

 **Source Code:** <https://github.com/adlernunez/Adler-Entri/tree/master/ARM%20F401xx/FINAL%20PROJECT/STM32F446RE>

Application Layer (main.c snippet)

```
``c
#include "stm32f401re_spi_driver.h"
#include <string.h>

int main(void) {
    SPI_Handle_t spi1_handle = {0};
    spi1_handle.pSPIx = SPI1;
```

```
spi1_handle.SPI_Config.SPI_Device_Mode = SPI_DEVICE_MODE_MASTER;
spi1_handle.SPI_Config.SPI_Bus_Config = SPI_BUS_CONFIG_FD;
spi1_handle.SPI_Config.SPI_DataFrame = SPI_DFF_8BIT;
spi1_handle.SPI_Config.SPI_CPHA = SPI_CPHA_LOW;
spi1_handle.SPI_Config.SPI_CPOL = SPI_CPOL_LOW;
spi1_handle.SPI_Config.SPI_SSM = SPI_SSM_EN;
spi1_handle.SPI_Config.SPI_CLK_Speed = SPI_PRESCALAR_128;
SPI_Init(&spi1_handle);
SPI_PeripheralControl(SPI1, ENABLE);
```

```
char data_to_send[] = "Temp: 34 C";
```

```
while(1) {
    // 1. Pull Chip Select LOW manually
    GPIO_WriteToOutputPin(GPIOA, 4, 0);

    // 2. Transmit data + null terminator
    uint8_t datalen = strlen(data_to_send) + 1;
    SPI_Transmit_Buffer(SPI1, (uint8_t*)data_to_send, datalen);

    // 3. Pull Chip Select HIGH
    GPIO_WriteToOutputPin(GPIOA, 4, 1);

    delay_ms(2000);
}
}
```

DEMO: <https://drive.google.com/drive/folders/1eXq7sDkj5dFtnOclj5tX9Nme-Wcqtehi?usp=sharing>